# RISK 231: SQL

Himchan Jeong, FSA

Department of Statistics & Actuarial Science
Simon Fraser University

# Recap of Codecademy and Joins

# Review: Manipulation

- **SQL** is a programming language designed to manipulate and manage data stored in relational databases.
  - A **relational database** is a database that organizes information into one or more tables.
  - A **table** is a collection of data organized into rows and columns.

A **statement** is a string of characters that the database recognizes as a valid command.

- `CREATE TABLE` creates a new table.
- `INSERT INTO` adds a new row to a table.
- `SELECT` queries data from a table.
- `ALTER TABLE` changes an existing table.
- `UPDATE` edits a row in a table.
- `DELETE FROM` deletes rows from a table.

# Review: Queries

- `SELECT` is the clause we use every time we want to query information from a database.
- `AS` renames a column or table.
- `DISTINCT` returns unique values.
- `WHERE` is a popular command that lets you filter the results of the query based on conditions that you specify.
- `LIKE` and `BETWEEN` are special operators that work with numbers, text, or date data types.
- `AND` and `OR` combine multiple conditions.
- `ORDER BY` sorts the result.
- `LIMIT` specifies the maximum number of rows that the query will return.
- `CASE` creates different outputs.

# Review: Aggregate functions

- `COUNT()`: count the number of rows
- `SUM()`: the sum of the values in a column
- `MAX()` / `MIN()`: the largest/smallest value
- `AVG()`: the average of the values in a column
- `ROUND()`: round the values in the column

**Aggregate functions** combine multiple rows together to form a single value of more meaningful information.

- `GROUP BY` is a clause used with aggregate functions to combine data from one or more columns.
- `HAVING` limits the results of a query based on an aggregate property.

# Review: Multiple tables

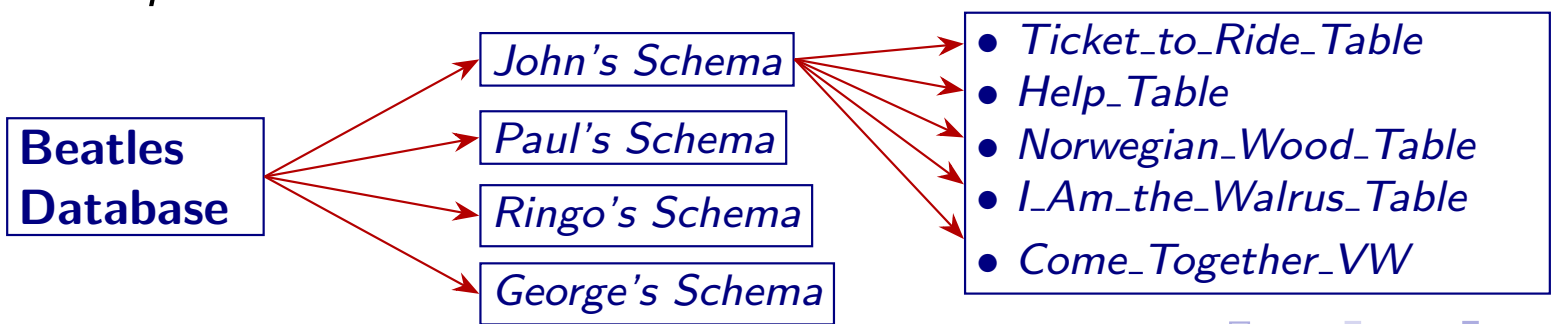
- `JOIN` will combine rows from different tables if the join condition is true.
- `LEFT JOIN` will return every row in the left table, and if the join condition is not met, `NULL` values are used to fill in the columns from the right table.
- **Primary key** is a column that serves as a unique identifier for the rows in the table.
- **Foreign key** is a column that contains the primary key of another table.
- `CROSS JOIN` lets us combine all rows of one table with all rows of another table.
- `UNION` stacks one dataset on top of another.
- `WITH` allows us to define one or more temporary tables that can be used in the final query.

# Databases & Schemas

- **Database:** an organized collection of data.
- **Schema:** the organization of rule sets that define the structure and relationship of data within a database.
- Within a database you may have multiple schemas; schemas can have multiple **tables** (datasets of a particular type or for a particular purpose).
- **Views** (`_VW`) are easy ways to execute a commonly used query.



*For example:*



# SQL Editor

**Let's take a look at the SQL App**

# SQL Order of Execution

Unlike some query languages, SQL does not execute its statements top-down; the clauses are executed in a particular order:

```
SELECT (5) DISTINCT ...                                  (6)
FROM ...                                                 (1)
JOIN ... ON ...                                          (1)
WHERE ...                                                (2)
GROUP BY ...                                             (3)
HAVING ...                                               (4)
ORDER BY ...                                             (7)
LIMIT/TOP (PL/SQL: WHERE ROWNUM <= #) ...                (8)
;
```

# SQL Commenting

Two ways to comment in SQL:

- Double dashes at the front of the line: `-- comment`
- Forward slash asterisk on each side of the comment:

  `/* comment */`

For example:

```
--Code to help look up some tables in the database

SELECT * FROM IRIS   /*Special table*/
 WHERE ID > 477156   /*Filters the data*/

--Cool, right?
```

# SQL Recap: Practice

Answer the following by writing SQL code in the SQL App:

## Exercise

1. Using the `COUNT()` function, determine how many rows are in the `SCHEMA_A.iris` table.
2. From the `SCHEMA_A.mtcars` table, select all rows that are not `cyl` 6 and have an `hp` greater than or equal to 105.
3. Using the `SUM()` function, display the sum of `len` for each level of `supp` in the `SCHEMA_B.ToothGrowth` table.

# SQL Recap: Practice – Answers

1. Number of rows in the iris table: **150**

```
SELECT COUNT(*) FROM SCHEMA_A.IRIS
```

2. Rows of mtcars that are not `cyl` 6 and have `hp` $\geq$ 105:

```
SELECT * FROM SCHEMA_A.MTCARS
WHERE CYL <> 6
AND HP >= 105
```

3. Sum of `len` for each level of `supp`: **OJ = 619.9 and VC = 508.9**

```
SELECT SUPP, SUM(LEN) FROM SCHEMA_B.TOOTHGROWTH
GROUP BY SUPP
```

# Table Management: Creating and Dropping Tables

- `CREATE TABLE` – constructs a table in a given schema and database

```
CREATE TABLE customers (customer_id number(10),
                        customer_name varchar2(50),
CONSTRAINT customers_pk PRIMARY KEY (customer_id));
```

- `CREATE TABLE AS` – constructs a table from a query

```
CREATE TABLE customers AS
    (SELECT * FROM SCHEMA.TABLE)
```

- `DROP TABLE` – removes a table in a given schema and database

```
DROP TABLE customers;
```

# Table Management: Adding Columns and Rows

- `ALTER TABLE` – add, modify, or drop/delete columns in a table

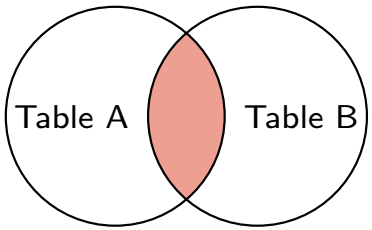
```
ALTER TABLE customers ADD (city varchar2(40)
                           DEFAULT 'Seattle');
```

- `INSERT INTO` – insert a single record or multiple records into a table

```
INSERT INTO customers
(customer_id, customer_name)
VALUES
(123456, 'HUSKY');
```

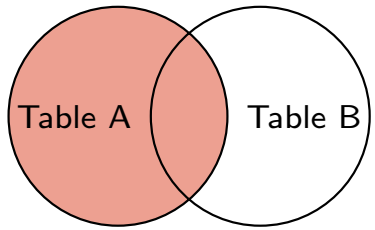
# Table Management: Joining Tables

**Inner Join**



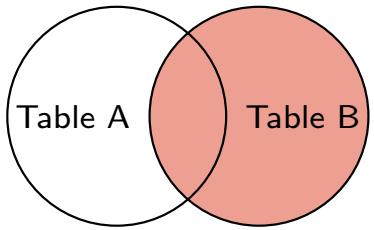

Select all records from Table A and Table B, where the join condition is met.

**Left Join**



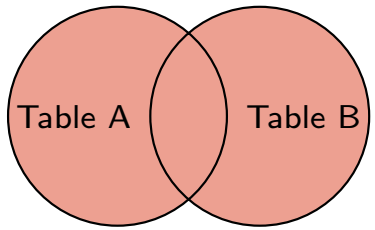

Select all records from Table A, along with records from Table B for which the join condition is met (if at all).

**Right Join**




Select all records from Table B, along with records from Table A for which the join condition is met (if at all).

**Full Join**




Select all records from Table A and Table B, regardless of whether the join condition is met or not.

# Joining Tables: Example Data

Throughout the join examples we use the following two tables:

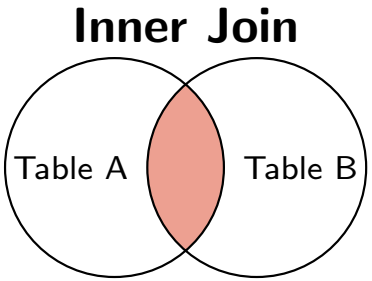
**Students**

| Student_Name | Grad_Year | Class | Grade |
|---|---|---|---|
| Patrick | 9 | Lunch | A+ |
| Sandy | 11 | Gym | A |
| Gary | 9 | Civics | B+ |
| Pearl | 12 | Calculus | B |
| Karen | 12 | Calculus | C- |
| Larry | 10 | Spelling | B |
| Plankton | 11 | Gym | B- |
| Squidward | 10 | Biology | A- |
| Bob | 11 | Biology | B+ |

**Teachers**

| Teacher_Name | Class |
|---|---|
| N. Watterson | Calculus |
| B. Joe | Gym |
| T. Wilson | Spelling |
| C. Krueger | Biology |
| S. Small | Civics |
| R. Robinson | Drama |
| B. Parham | Band |
| G. Watterson | Music Theory |

# Joining Tables: INNER JOIN

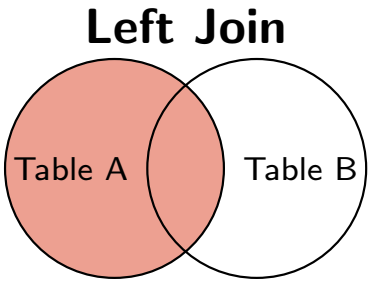



```
SELECT
    students.*,
    teachers.Teacher_Name
FROM students INNER JOIN teachers
    ON students.Class = teachers.Class;
```

**Inner Join**

| Student_Name | Grad_Year | Class | Grade | Teacher_Name |
|---|---|---|---|---|
| Sandy | 11 | Gym | A | B. Joe |
| Gary | 9 | Civics | B+ | S. Small |
| Pearl | 12 | Calculus | B | N. Watterson |
| Karen | 12 | Calculus | C- | N. Watterson |
| Larry | 10 | Spelling | B | T. Wilson |
| Plankton | 11 | Gym | B- | B. Joe |
| Squidward | 10 | Biology | A- | C. Krueger |
| Bob | 11 | Biology | B+ | C. Krueger |

# Joining Tables: LEFT JOIN



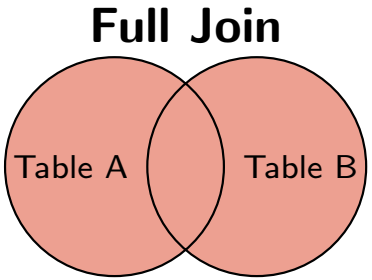

```
SELECT
    students.*,
    teachers.Teacher_Name
FROM students LEFT JOIN teachers
    ON students.Class = teachers.Class;
```

**Left Join**

| Student_Name | Grad_Year | Class | Grade | Teacher_Name |
|---|---|---|---|---|
| Patrick | 9 | Lunch | A+ | (blank) |
| Sandy | 11 | Gym | A | B. Joe |
| Gary | 9 | Civics | B+ | S. Small |
| Pearl | 12 | Calculus | B | N. Watterson |
| Karen | 12 | Calculus | C- | N. Watterson |
| Larry | 10 | Spelling | B | T. Wilson |
| Plankton | 11 | Gym | B- | B. Joe |
| Squidward | 10 | Biology | A- | C. Krueger |
| Bob | 11 | Biology | B+ | C. Krueger |

Note: there is no match in the Teachers table for Lunch.

# Joining Tables: FULL JOIN




```
SELECT
    students.*,
    teachers.*
FROM students FULL JOIN teachers
    ON students.Class = teachers.Class;
```

**Full Join**

| Student_Name | Grad_Year | Class | Grade | Teacher_Name | Class1 |
|---|---|---|---|---|---|
| Patrick | 9 | Lunch | A+ | | |
| Sandy | 11 | Gym | A | B. Joe | Gym |
| Gary | 9 | Civics | B+ | S. Small | Civics |
| Pearl | 12 | Calculus | B | N. Watterson | Calculus |
| Karen | 12 | Calculus | C- | N. Watterson | Calculus |
| Larry | 10 | Spelling | B | T. Wilson | Spelling |
| Plankton | 11 | Gym | B- | B. Joe | Gym |
| Squidward | 10 | Biology | A- | C. Krueger | Biology |
| Bob | 11 | Biology | B+ | C. Krueger | Biology |
| | | | | R. Robinson | Drama |
| | | | | B. Parham | Band |
| | | | | G. Watterson | Music Theory |

Note: both tables have a `Class` column, so a new name is given in the joined table: `Class1`. Also, unmatched items are given blank values in the joined table.

# Table Management: Practice

## Exercise

1. Using a `LEFT JOIN`, append `UrbanPop` and `Theft` from the `SCHEMA_C.USArrests` table to the `SCHEMA_A.Iris` table using `ID`.

2. Using an `INNER JOIN`, combine the `SCHEMA_C.USArrests` and `SCHEMA_B.ToothGrowth` tables using `ID`.

# Table Management: Practice – Answers

1. Using a `LEFT JOIN`, append `UrbanPop` and `Theft` from `SCHEMA_C.USArrests` to `SCHEMA_A.Iris` using `ID`.

```
SELECT IRIS.*, USARRESTS.URBANPOP, USARRESTS.THEFT
FROM SCHEMA_A.IRIS
LEFT JOIN SCHEMA_C.USARRESTS
    ON SCHEMA_A.IRIS.ID = SCHEMA_C.USARRESTS.ID
```

2. Using an `INNER JOIN`, combine `SCHEMA_C.USArrests` and `SCHEMA_B.ToothGrowth` using `ID`.

```
SELECT * FROM SCHEMA_C.USARRESTS
INNER JOIN SCHEMA_B.TOOTHGROWTH
    ON SCHEMA_C.USARRESTS.ID = SCHEMA_B.TOOTHGROWTH.ID
```

# Note: Three Ways to Write an `INNER JOIN`

In SQL, you can do an *INNER JOIN* (answer #2) in several ways:

```
SELECT * FROM SCHEMA_C.USARRESTS
    INNER JOIN SCHEMA_B.TOOTHGROWTH
    ON SCHEMA_C.USARRESTS.ID = SCHEMA_B.TOOTHGROWTH.ID
```

*or*

```
SELECT * FROM SCHEMA_C.USARRESTS
    JOIN SCHEMA_B.TOOTHGROWTH
    ON SCHEMA_C.USARRESTS.ID = SCHEMA_B.TOOTHGROWTH.ID
```

*or*

```
SELECT * FROM SCHEMA_C.USARRESTS,
              SCHEMA_B.TOOTHGROWTH
    WHERE SCHEMA_C.USARRESTS.ID = SCHEMA_B.TOOTHGROWTH.ID
```

# Aliasing and CASE WHEN Statements

# Aliasing

- Reference tables or subqueries in code with a shortcut.
- Aliases allow you to refer to tables multiple times in your code with a shortcut, which gives you much cleaner code.

*e.g. Instead of using*

```
SELECT
    students.*,
    teachers.Teacher_Name
FROM students LEFT JOIN teachers
    ON students.Class = teachers.Class;
```

*We can use*

```
SELECT
    s.*,
    t.Teacher_Name
FROM students s LEFT JOIN teachers t
    ON s.Class = t.Class;
```

# Aliasing De-clutters Your Code

Aliasing can be **VERY** useful in de-cluttering your code!

NOTE: Use abbreviations that tie back to your table names! Try not to use a, b, c, T1, T2, ... to alias.

*e.g. Instead of using*

```
SELECT FAKE_SCHEMA.EXAMPLE_DATA_VW.POL_NAME,
FAKE_SCHEMA.EXAMPLE_DATA_VW.POL_EXPIRATION, FAKE_SCHEMA_TWO.EXAMPLE_DATA_TABLE.POL_NUMBER
FROM FAKE_SCHEMA.EXAMPLE_DATA_VW LEFT JOIN FAKE_SCHEMA_TWO.EXAMPLE_DATA_TABLE ON
FAKE_SCHEMA.EXAMPLE_DATA_VW.POL_KEY = FAKE_SCHEMA_TWO.EXAMPLE_DATA_TABLE.POL_KEY AND
FAKE_SCHEMA.EXAMPLE_DATA_VW.NEW_RENEWAL_CODE = FAKE_SCHEMA_TWO.EXAMPLE_DATA_TABLE.
      NEW_RENEWAL_CODE
```

*We can use*

```
SELECT EDV.POL_NAME, EDV.POL_EXPIRATION, EDT.POL_NUMBER
FROM FAKE_SCHEMA.EXAMPLE_DATA_VW EDV
     LEFT JOIN FAKE_SCHEMA_TWO.EXAMPLE_DATA_TABLE EDT
          ON EDV.POL_KEY = EDT.POL_KEY
          AND EDV.NEW_RENEWAL_CODE = EDT.NEW_RENEWAL_CODE
```

# Aliasing: Practice – *use aliasing!*

## Exercise

1. Using a `LEFT JOIN`, append `weight` from the `SCHEMA_B.PlantGrowth` table to the `SCHEMA_A.mtcars` table using `ID` and `color_ID`, selecting `Name`, `mpg`, `cyl`, and `wt` from the `mtcars` table. Also, create a new column called `weight_diff` that gives the difference of the `weight` column and the `wt` column.

2. Using `INNER JOIN`, combine the `SCHEMA_A.Iris`, `SCHEMA_B.PlantGrowth`, and `SCHEMA_C.USArrests` tables using `ID` for `iris` and `PlantGrowth`, and `State` for `iris` and `USArrests`. Select all of the data in the `iris` table, the `weight` column from the `PlantGrowth` table, and the `Murder`, `State`, and `Assault` columns from the `USArrests` table.

# Aliasing: Practice – Answers (1)

1. `LEFT JOIN` `weight` from `SCHEMA_B.PlantGrowth` onto `SCHEMA_A.mtcars` (`color_ID` $\leftrightarrow$ `ID`), selecting `Name`, `mpg`, `cyl`, `wt`, and the new column `weight_diff` = `weight` − `wt`.

```
SELECT
    M.NAME,
    M.MPG,
    M.CYL,
    M.WT,
    P.WEIGHT,
    (P.WEIGHT - M.WT) AS WEIGHT_DIFF
FROM SCHEMA_A.MTCARS M
        LEFT JOIN SCHEMA_B.PLANTGROWTH P
            ON M.COLOR_ID = P.ID
```

# Aliasing: Practice – Answers (2)

2. INNER JOIN of SCHEMA_A.Iris, SCHEMA_B.PlantGrowth (on ID) and SCHEMA_C.USArrests (on State): all of iris, weight from PlantGrowth, and Murder, State, Assault from USArrests.

```
SELECT
    I.*,
    P.WEIGHT,
    U.MURDER,
    U.STATE,
    U.ASSAULT
FROM SCHEMA_A.IRIS I,
     SCHEMA_B.PLANTGROWTH P,
     SCHEMA_C.USARRESTS U
WHERE I.ID = P.ID AND I.STATE = U.STATE
```

# String Handling (1)

- `SUBSTR(x, start [, length]);`
  - Returns a substring of `x` that begins at the position specified by `start`. An optional `length` for the substring may be supplied.
  - `start` can be negative to count from the end of the string.

*policy=88UECAB4758* `SUBSTR(policy,6,6)` = *AB4758*
*policy=88UECAB4758* `SUBSTR(policy,-4)` = *4758*

- `CONCAT(x, y);`
  - Concatenates the strings `x` and `y` and returns the appended string.
  - You can also use `||` to concatenate strings.

*pol_sym=UEC* `CONCAT(pol_sym,pol_num)` = *UECAB4758* *or*
*pol_num=AB4758* `(pol_sym || pol_num)` = *UECAB4758*

- `LENGTH(x);`
  - Returns the number of characters in `x`.

*policy=88UECAB4758* `LENGTH(policy)` = *11*

# String Handling (2)

- `NVL(x, value);`
  - Returns `value` if `x` is null; otherwise, `x` is returned. e.g. `NVL(prem,0)`
- `LTRIM(x [, trim_string]);` / `RTRIM(x [, trim_string]);`
  - Trims characters from the left or right of `x` (by default, leading/trailing spaces).
- `CHARINDEX(x, search_string);`
  - Searches `x` for `search_string` and returns the position.
- `REPLACE(x, search_string, replace_string);`
  - Searches `x` for `search_string` and replaces it with `replace_string`.

# String Handling: Practice

## Exercise

Write a query that extracts two columns from the `SCHEMA_A.mtcars` table, which decomposes the `Name` into two parts (separated by the first space): `Brand` and `Model`.

## Exercise

Write a query that extracts the accident year, month, and day from `acc_dt` of the `SCHEMA_D.lossexpense` table. Further, create a column with date format using the extracted information.

## Exercise

Using a `LEFT JOIN`, append all columns from the `SCHEMA_D.premexposures` table to the `SCHEMA_D.lossexpense` table using `pol_id`. Note that `pol_id` can be obtained in the `SCHEMA_D.lossexpense` table by concatenating `pol_regsym` and `pol_num`.

# String Handling: Practice – Answers

## Split Name of mtcars into Brand and Model:

```
SELECT mtcars.Name,
    SUBSTR(mtcars.Name, 1, CHARINDEX(' ', mtcars.Name)-1) AS Brand,
    SUBSTR(mtcars.Name, CHARINDEX(' ', mtcars.Name),
           LENGTH(mtcars.Name)) AS Model
FROM mtcars;
```

Extract accident year, month, and day from acc_dt (format m/d/yyyy):

```
SELECT le.acc_dt,
    SUBSTR(le.acc_dt, -4) AS year,
    SUBSTR(le.acc_dt, 1, CHARINDEX('/', le.acc_dt)-1) AS month,
    SUBSTR(SUBSTR(le.acc_dt, CHARINDEX('/', le.acc_dt)+1), 1,
           CHARINDEX('/', SUBSTR(le.acc_dt, CHARINDEX('/', le.acc_dt)
     +1))-1) AS day
FROM Schema_D.lossexpense le;
```

Append premexposures to lossexpense via the concatenated pol_id:

```
SELECT le.*, pe.*
FROM SCHEMA_D.lossexpense le LEFT JOIN SCHEMA_D.premexposures pe
    ON CONCAT(le.pol_regsym, le.pol_num) = pe.pol_id;
```

# CASE Statement

- Effectively an IF-ELSE statement.
- CASE looks for the first match; once it meets this match, it performs the THEN and goes to the next row of the data.

```
SELECT c.*,
       CASE
          WHEN c.city = 'Seattle' THEN 'Northwest'
          WHEN c.city = 'Orlando' THEN 'Southeast'
          ELSE 'Merica'
       END AS REGION
  FROM customers c
```

- Use an ELSE statement when you want the remaining circumstances to be something other than blank.
- Don't forget the END statement!
- AS REGION names the column being created.

# CASE Statement: Practice

## Exercise

Write a query that creates an additional column in the `SCHEMA_C.USArrests` table using a `CASE` statement to give unique codes based on the values in the `Murder` column. For example, if the `Murder` rate is greater than 10, the new column could be '15+' or 'High' or 'Terrifying!'.

# CASE Statement: Practice

## Exercise

Write a query that creates an additional column in the SCHEMA_C.USArrests table using a CASE statement to give unique codes based on the values in the Murder column. For example, if the Murder rate is greater than 10, the new column could be '15+' or 'High' or 'Terrifying!'.

**Answer:**

```
SELECT
    U.*,
    CASE WHEN U.MURDER > 15 THEN 'AHHHHH!!!!'
         WHEN U.MURDER > 10 THEN 'SCARY AT NIGHT!'
         WHEN U.MURDER > 5 THEN 'JUST BE CAREFUL'
         ELSE 'PEACE, LOVE, AND HAPPINESS'
    END AS MURDER_CODE
FROM SCHEMA_C.USARRESTS U
```

# Nested Queries and WITH Statements

# Subqueries: Motivation

- Oftentimes, you will need more than one table in more than one step to get what you need.
- What if you want to accomplish this without creating multiple tables in the database? Or. . . what if you want to accomplish this without creating mid-step tables?

For example:

```
CREATE TABLE Average_Age AS
SELECT
    AVG(AGE) AS AvAge
FROM SCHEMA.TABLE;

CREATE TABLE Greater_than_Average AS
SELECT
    NAMES
FROM SCHEMA.TABLE nm, Average_Age aa
WHERE nm.AGE > aa.AvAge;
```

# Subquery

- SQL query nested in a larger query
  - Can be added to the *WHERE*, *SELECT*, and *FROM* clauses
  - Can be run on their own to produce results, often single or multiple values
- Queries with subqueries execute from the innermost query outward
- Multiple nested subqueries are possible

```
SELECT NAMES
  FROM SCHEMA.TABLE
  WHERE AGE > (SELECT AVG(AGE) FROM SCHEMA.TABLE);
```

**Outer Query** **Inner Query**

# Subquery: Practice

## Exercise

Using a subquery, return all rows from the `iris` table with `ID`s the same as the `color_ID`s associated with Mercedes found in the `mtcars` table. *HINT: the wildcard character in SQL is %.*

# Subquery: Practice

## Exercise

Using a subquery, return all rows from the `iris` table with `ID`s the same as the `color_ID`s associated with Mercedes found in the `mtcars` table. *HINT: the wildcard character in SQL is %.*

**Answer:**

```
SELECT * FROM IRIS
    WHERE ID IN (SELECT COLOR_ID FROM MTCARS
                        WHERE NAME LIKE 'Merc%')
```

# Subquery: Scoping Considerations

- Be mindful of the order of operations; the scope of any query is contained in the **FROM** clause.
- Any subquery in the *WHERE* clause of an outer query will return values and not a table that can be selected from.

*For example: we cannot pull AVG_AGE in the outer query from this inner query* ×

```
SELECT NAMES, AVG_AGE
FROM SCHEMA.TABLE
WHERE AGE > (SELECT AVG(AGE) AS AVG_AGE FROM SCHEMA.TABLE);
```

*This code will work correctly* ✓

```
SELECT sch.NAMES, iq.AVG_AGE
FROM SCHEMA.TABLE sch,
     (SELECT AVG(AGE) AS AVG_AGE FROM SCHEMA.TABLE) iq
WHERE sch.AGE > iq.AVG_AGE;
```

# Subquery: `WITH`

- Special subquery that generates a table to be used within a larger query.
- Very useful for organizing complex queries (**MUCH** easier to read than nested queries, particularly when there are multiple subqueries!)

*For example:*

```
WITH temp AS (SELECT AVG(AGE) AS AVG_AGE FROM SCHEMA.TABLE)
SELECT sch.NAMES, iq.AVG_AGE
FROM SCHEMA.TABLE sch, temp iq
WHERE sch.AGE > iq.AVG_AGE;
```

```
SELECT sch.NAMES, iq.AVG_AGE
FROM SCHEMA.TABLE sch,
     (SELECT AVG(AGE) AS AVG_AGE FROM SCHEMA.TABLE) iq
WHERE sch.AGE > iq.AVG_AGE;
```

*Both result in the same output, but the top one is easier to read.*

# WITH: Practice

## Exercise

Using a `WITH` statement, return all rows from the `iris` table with `ID`s the same as the `color_ID`s associated with Mercedes found in the `mtcars` table. *HINT: the wildcard character in SQL is %.*

# WITH: Practice

## Exercise

Using a `WITH` statement, return all rows from the `iris` table with `ID`s the same as the `color_ID`s associated with Mercedes found in the `mtcars` table. *HINT: the wildcard character in SQL is %.*

**Answer:**

```
WITH TEMP AS
(SELECT COLOR_ID FROM MTCARS
        WHERE NAME LIKE 'Merc%')
SELECT I.* FROM IRIS I, TEMP T
    WHERE I.ID = T.COLOR_ID
```

# Subquery and `WITH`: Practice

## Exercise

Return all rows from the `ToothGrowth` table along with the `Murder` column from the `USArrests` table for urban populations greater than 80.

Do this using a subquery, and again using a `WITH` statement.

# Subquery and WITH: Practice – Answers

## Using a subquery:

```
SELECT
    T.*, P.MURDER
FROM TOOTHGROWTH T
INNER JOIN (SELECT U.*
            FROM USARRESTS U
            WHERE U.URBANPOP > 80) P
    ON T.ID = P.ID
```

## Using a WITH statement:

```
WITH POP AS (SELECT U.*
             FROM USARRESTS U WHERE U.URBANPOP > 80)
SELECT
    T.*, P.MURDER
FROM TOOTHGROWTH T
INNER JOIN POP P
    ON T.ID = P.ID
```